DeltaFi Action Kit: Java vs Go Comparative Analysis

Based on deltafi-java-hello-world and deltafi-go-hello-world reference plugins

Overview: Both action kits provide the same capabilities — transform, split, join, egress, and timed ingress actions with support for content manipulation, metadata, annotations, lookup tables, and artifact management. The Java kit uses Spring Boot with annotation-based registration and class inheritance. The Go kit uses typed generic registration functions with a Deltafile-centric API.

1. Plugin Bootstrap

Java

```
@SpringBootApplication
public class HelloWorld {
    public static void main(String[] args) {
        SpringApplication.run(HelloWorld.class, args);
    }
}
```

Go

```
func main() {
    actionkit.RunPlugin(&actionkit.PluginConfig{
        Coordinates: &actionkit.PluginCoordinates{
            GroupID:    "org.deltafi",
            ArtifactID: "deltafi-go-hello-world",
            Version:    version,
        },
        Domain:      "org.deltafi.go-hello-world",
        Description: description,
    })
}
```

Key difference: Java uses Spring Boot auto-configuration and classpath scanning. Go uses explicit plugin configuration with RunPlugin(). Actions self-register via init() functions in Go, vs @Component annotation in Java.

2. Action Registration

Java

```
@Component
public class HelloWorldJavaTransformAction
    extends TransformAction<HelloWorldJavaTransformParameters> {

    public HelloWorldJavaTransformAction(...) {
        super("Add some content noting that we did a really good job");
    }
}
```

Go

```
//go:embed transform_action.yaml
var transformActionOptions string

func init() {
    actionkit.RegisterTransform(
        "HelloWorldGoTransformAction",
        &HelloWorldGoTransform{}).
        WithParams(HelloWorldGoTransformParams{}).
        WithOptionsYAML(transformActionOptions)
}
```

Key difference: Java uses class inheritance (extends TransformAction<P>) to declare the action type and bind the params type. Go uses typed generic registration functions (RegisterTransform, RegisterEgress, RegisterIngress, RegisterJoinTransform) — the compiler enforces the method signature via interface constraints and infers the params type. Action options are declared in an embedded YAML file in Go.

3. Parameter Declaration

Java

```
@Data
@EqualsAndHashCode(callSuper = true)
public class HelloWorldJavaTransformParameters
    extends ActionParameters {

    @JsonProperty(defaultValue = "false")
    @JsonPropertyDescription("If TRUE, convert to uppercase")
    private boolean convertToUpper = false;

    @JsonProperty(defaultValue = "44")
    @JsonPropertyDescription("DID prefix for errors")
    private String didErrorPrefix = "44";
}
```

Go

```
type HelloWorldGoTransformParams struct {
    ConvertToUpper bool   `json:"convertToUpper"
                          description:"If true, convert to uppercase"
                          default:"false"`
    DidErrorPrefix string `json:"didErrorPrefix"
                          description:"DID prefix for errors"
                          default:"44"`
}
```

Key difference: Java uses Lombok @Data for getters/setters and Jackson annotations for schema metadata. Go uses struct tags (description, default) — no code generation or annotation processing needed.

4. Transform Action

Java

```
@Override
public TransformResultType transform(
    @NotNull ActionContext context,
    @NotNull HelloWorldJavaTransformParameters params,
    @NotNull TransformInput input) {

  // Filter
  if (context.getDid().toString()
      .startsWith(params.getDidFilterPrefix())) {
    return new FilterResult(context,
        "Filter due to did_filter_prefix match");
  }

  // Error
  if (context.getDid().toString()
      .startsWith(params.getDidErrorPrefix())) {
    return new ErrorResult(context,
        "Error due to did_error_prefix match",
        params.getDidErrorPrefix() + " detected");
  }

  // Load content
  String data = input.getContent().getFirst()
      .loadString();

  // Save new content
  TransformResult result = new TransformResult(context);
  ActionContent content = result.saveContent(
      data.getBytes(), "name", MediaType.TEXT_PLAIN);
  content.addTag("hello-file");

  // Metadata & annotations
  result.addMetadata("key", "value");
  result.addAnnotation("language", "English");

  return result;
}
```

Go

```
func (a *HelloWorldGoTransform) Transform(
    df *actionkit.Deltafile,
    params HelloWorldGoTransformParams,
) *actionkit.Deltafile {

  // Filter
  if strings.HasPrefix(df.Did,
      params.DidFilterPrefix) {
    return df.Filterf(
        "Filter due to did_filter_prefix match")
  }

  // Error
  if strings.HasPrefix(df.Did,
      params.DidErrorPrefix) {
    return df.Errorf(
        "Error due to did_error_prefix match").
        SetErrorContext(
            params.DidErrorPrefix + " detected")
  }

  // Load content
  content, _ := df.ContentAt(0)
  data, _ := content.LoadString()

  // Save new content
  newContent, _ := df.SaveStringContent(
      data, "name", "text/plain")
  newContent.AddTag("hello-file")

  // Metadata & annotations
  return df.
      AddMetadata("key", "value").
      AddAnnotation("language", "English")
}
```

Key differences:
• Java: Separate result types (TransformResult, ErrorResult, FilterResult) returned via a TransformResultType union. ActionContext passed explicitly.
• Go: The Deltafile IS the result — error/filter state is set on it via Errorf()/Filterf(). All methods are chainable. No separate context object; context fields live on the Deltafile (df.Did, df.Name, etc.). Params are typed and passed directly.

5. Split / Multi-Child Transform

Java

```
@Override
public TransformResultType transform(
    @NotNull ActionContext context,
    @NotNull Params params,
    @NotNull TransformInput input) {

  TransformResults manyResults =
      new TransformResults(context);

  String data = input.getContent()
      .getFirst().loadString();
  ActionContent content =
      ActionContent.saveContent(context,
          data.getBytes(), "content", "text/plain");

  for (int n = 0; n < params.getNumberOfChildren(); n++) {
    String name = context.getDeltaFileName() + "-" + n;
    ChildTransformResult child =
        new ChildTransformResult(context, name);
    child.addContent(content);
    child.addMetadata("key", "value");
    child.addAnnotation("childIndex", "" + n);
    manyResults.add(child);
  }

  return manyResults;
}
```

Go

```
func (a *HelloWorldGoSplitTransform) Transform(
    df *actionkit.Deltafile,
    params HelloWorldGoSplitTransformParams,
) *actionkit.Deltafile {

  content, err := df.ContentAt(0)
  if err != nil {
    return df.Errorf("No content")
  }
  data, _ := content.LoadString()

  for n := range params.NumberOfChildren {
    name := fmt.Sprintf("%s-%d", df.Name, n)
    child := df.NewChild(name)
    child.SaveStringContent(
        data, "content", "text/plain")
    child.AddMetadata("key", "value").
        AddAnnotation("childIndex", strconv.Itoa(n))
  }

  return df
}
```

Key difference: Both kits use the same Transform method for splits. Java returns a TransformResults container. Go calls df.NewChild(name) to create children — the kit detects children and produces a multi-child transform event automatically. An action can decide at runtime whether to split or not based on the input.

6. Join Action

Java

```
// Implements Join interface
@Override
public TransformInput join(
    List<TransformInput> inputs) {

  List<ActionContent> content = new ArrayList<>();
  Map<String, String> metadata = new HashMap<>();
  for (TransformInput input : inputs) {
    content.addAll(input.getContent());
    metadata.putAll(input.getMetadata());
  }
  return TransformInput.builder()
      .content(content)
      .metadata(metadata)
      .build();
}

@Override
public TransformResultType transform(
    ActionContext context, Params params,
    TransformInput input) {
  // Process joined input...
}
```

Go

```
func (a *HelloWorldGoJoiningTransform) Join(
    inputs []*actionkit.Deltafile,
    joined *actionkit.Deltafile,
    params HelloWorldGoJoiningTransformParams,
) *actionkit.Deltafile {

  for _, input := range inputs {
    joined.AddContentSlice(input.GetContentSlice())
    joined.AddMetadataMap(input.GetMetadataMap())
  }
  return joined
}

func (a *HelloWorldGoJoiningTransform) Transform(
    df *actionkit.Deltafile,
    params HelloWorldGoJoiningTransformParams,
) *actionkit.Deltafile {
  // Process joined input...
}
```

7. Egress Action

Java

```
@Override
public EgressResultType egress(
    @NotNull ActionContext context,
    @NotNull ActionParameters params,
    @NotNull EgressInput input) {

  String data = new String(
      input.getContent().loadBytes(),
      StandardCharsets.UTF_8);

  if (data.contains("error")) {
    return new ErrorResult(context,
        "Failed to egress",
        new Exception("error in content"));
  }

  return new EgressResult(context);
}
```

Go

```
func (a *HelloWorldGoEgress) Egress(
    df *actionkit.Deltafile,
    params struct{},
) *actionkit.Deltafile {

  content, _ := df.FirstContent()
  data, _ := content.LoadString()

  if strings.Contains(data, "error") {
    return df.Errorf("Failed to egress").
        SetErrorContext("error in content")
  }

  return df
}
```

8. Timed Ingress Action

Java

```
@Override
public IngressResultType ingress(
    @NotNull ActionContext context,
    @NotNull ActionParameters params) {

  int index = 0;
  if (context.getMemo() != null) {
    index = 1 + Integer.parseInt(
        context.getMemo());
  }

  String filename = context.getFlowName()
      + "-" + index;
  IngressResultItem item =
      new IngressResultItem(context, filename);
  item.saveContent("Content " + index,
      filename, "text/plain");
  item.addMetadata("index",
      String.valueOf(index));

  IngressResult result =
      new IngressResult(context);
  result.addItem(item);
  result.setMemo(String.valueOf(index));
  return result;
}
```

Go

```
func (a *HelloWorldGoTimedIngress) Ingress(
    input *actionkit.IngressInput,
    params struct{},
) *actionkit.IngressResult {

  index := 0
  if input.Memo != "" {
    if parsed, err := strconv.Atoi(
        input.Memo); err == nil {
      index = parsed + 1
    }
  }

  filename := fmt.Sprintf("%s-%d",
      input.FlowName, index)
  item := input.NewDeltafile(filename)
  item.SaveStringContent(
      fmt.Sprintf("Content %d", index),
      filename, "text/plain")
  item.AddMetadata("index",
      strconv.Itoa(index))

  return actionkit.NewIngressResult().
      AddItem(item).
      SetMemo(strconv.Itoa(index))
}
```

9. Lookup Table Queries

Java

```
// Injected via constructor
private final LookupTableClient lookupTableClient;

LookupResults colors = lookupTableClient.lookup(
    "java_scheduled_lookup_table",
    LookupOptions.builder()
        .matchingColumnValues(
            Map.of("color", Set.of("red", "green")))
        .resultColumns(Set.of("column_a"))
        .build());

for (Map<String, String> row : colors.results()) {
    String value = row.get("column_a");
}
```

Go

```
// Package-level function, no injection needed
results, err := actionkit.Lookup(
    "go_scheduled_lookup_table",
    actionkit.LookupQuery().
        WithMatch("color", "red", "green").
        WithResultColumns("column_a"))

for _, row := range results.Rows {
    value := row["column_a"]
}
```

10. Artifact Management

Java

```
// Injection + annotation
public HelloWorldJavaTransformAction(
    @PluginArtifact(
        artifactName = "metadata.json",
        defaultResource = "metadata.json")
    ArtifactHandle metadataResource) {

    metadataResource.onUpdate(this::handleUpdate);
}

// In-action artifact access
ArtifactHandle handle =
    artifactRegistryService.getArtifact("hello.txt");
byte[] data = handle.getBytes();
```

Go

```
// ConfigureArtifacts method (detected via interface)
func (a *MyAction) ConfigureArtifacts(
    client *actionkit.ArtifactClient) {

    client.OnUpdateFetchWithDefault(
        "metadata.json", "", defaults, callback)
}

// In-action artifact access (package-level)
fetched, err := actionkit.GetArtifact(
    "hello.txt", "")
data := fetched.Data
```

11. Comparative Summary

Aspect	Java	Go
Framework	Spring Boot (DI, annotations, classpath scan)	Standalone (init() self-registration, generic interface constraints)
Action typing	Class inheritance (extends TransformAction<P>)	Generic registration (RegisterTransform, RegisterEgress, etc.)

Params binding	Generic type parameter on base class	Inferred from method signature via generics
Params metadata	@JsonProperty + @JsonPropertyDescription	Struct tags: description:, default:
Result model	Separate result types (TransformResult, ErrorResult, etc.)	Single Deltafile with error/filter state
Context access	ActionContext parameter	Fields on Deltafile (df.Did)
Content ops	input.getContent().loadString()	content.LoadString() (context-free)
Lookup queries	Injected LookupTableClient	Package-level actionkit.Lookup()
Artifacts	@PluginArtifact annotation + DI	ConfigureArtifacts method + package-level functions
Action options	Spring annotations / programmatic	Embedded YAML file via //go:embed
Split actions	Same transform(), returns TransformResults	Same Transform(), calls df.NewChild()
Dispatch	Virtual method dispatch (interface call)	Typed closures via generics (zero reflection at invocation)
Compile-time safety	Generics + annotations enforce signatures	Generic interface constraints enforce signatures
Method chaining	On result objects only	All Deltafile methods are chainable
Binary size	~150MB+ (JVM + Spring + dependencies)	~15MB (single static binary)
Startup time	Several seconds (Spring context init)	Milliseconds